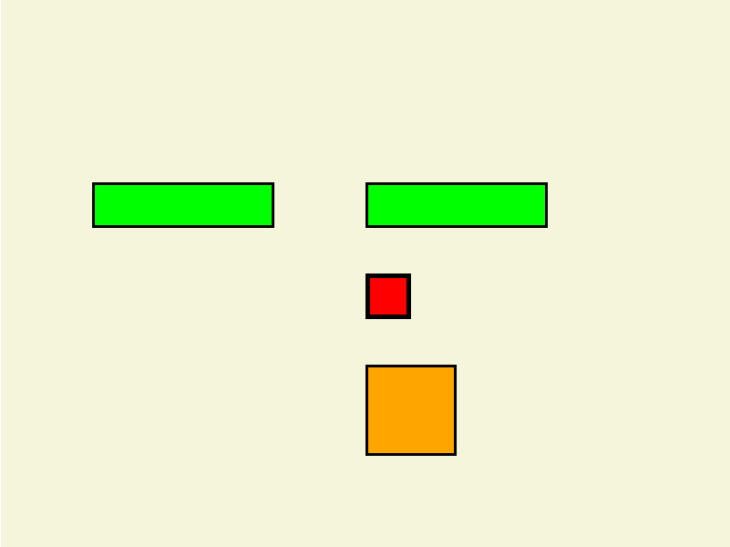
Barrier System

The Barrier System allows you to add and remove barriers and check for collisions.
Full Example:

```
from mckPy import *

addBarrier(100, 200, 200, 50, colour = "green")
addBarrier(400, 200, 200, 50, colour = "green")

addBarrier(400, 400, 100, 100, colour = "orange", kind = "fire")

x = 400
y = 300
while running():
    fill("beige")

    if leftKey() and hitBarrier(x-5, y, 50, 50) == False:
        x = x - 5
    if rightKey() and hitBarrier(x+5, y, 50, 50) == False:
        x = x + 5
    if upKey() and hitBarrier(x, y-5, 50, 50) == False:
        y = y - 5
    if downKey() and hitBarrier(x, y+5, 50, 50) == False:
        y = y + 5

    drawBarriers(outline = 3)
    if hitBarrier(x, y, 50, 50):
        drawRect("plum", x, y, 50, 50, outline = 5)
    elif hitBarrier(x, y, 50, 50, kind = "fire"):
        drawRect("yellow", x, y, 50, 50, outline = 5)
    else:
        drawRect("red", x, y, 50, 50, outline = 5)
```

This example lets you move the red box using the keyboard. You can use either arrows or WASD. You can't go on top of the green boxes and when you go on top of the orange box you change from red to yellow.



Key Features and Commands

Movement

The main character will be controlled with the keyboard. There are 5 commands in mckPy to check the status of keys on the keyboard.

`upKey()` – This is True when either the **w** key or the **up** arrow is pressed.
`leftKey()` – This is True when either the **a** key or the **left** arrow is pressed.
`downKey()` – This is True when either the **s** key or the **down** arrow is pressed.
`rightKey()` – This is True when either the **d** key or the **right** arrow is pressed.
`keyPressed(keyCode)` – This is True when the indicated key is pressed. Find all keycodes at: https://www.pygame.org/docs/ref/key.html

Interaction

When dealing with rectangles, there are 2 ways to check if rectangles are overlapping.

`overlaps(x1, y1, w1, h1, x2, y2, w2, h2)`

– The first 4 values represent rectangle 1, the next 4, rectangle 2. This function returns True when the two rectangles are overlapping and False otherwise.

Barriers

The Barrier system has a few important functions.

1. **addBarrier(x, y, w, h, colour = "black", kind = "wall", image=None)**
   a. You can add a rectangular barrier to your program with addBarrier, These barriers will be remembered and used when you call drawBarriers and hitBarriers.
   b. If you don't specify the colour, the barrier will be black.
   c. **kind:** You can specify the kind of your barrier. If you don't specify your barrier will be a wall. If you do specify, you can control which barriers get drawn or hit.
   d. **image:** If you use an image it will fill the barrier with that image. If the image is bigger than the barrier, it will take part of the image, otherwise it will stretch the image to fit the barrier.
   e. **tile:** This is like image, except the image given is tiled onto the barrier.

2. **drawBarriers(outline = 0, outlineColour=0, kind = "", debug=False)**
   Draws all the barriers. If you specify a kind, it will only draw the ones of that kind. If you set debug=True it will also print the number of each barrier based on the order they were added.

3. **hitBarrier(x, y, w, h, kind = "wall")**
   Returns True if the rectangle specified (x, y, w, h) overlaps with any barrier of the kind specified. If you do not specify the kind, it will check for walls.

4. **removeBarrier(x, y, w, h, kind = "wall")**
   Removes a barrier from the program. The rectangle specified (x, y, w, h) must match exactly.

5. **removeHitBarrier(x, y, w, h, kind = "wall")**

   Removes a barrier from the program. It removes the Barrier that hits the rectangle specified (x, y, w, h).

6. **clearBarriers(kind = None)**

   Removes all barriers from the program. If you specify a kind, it only removes the ones of that kind.